```
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include "MAX30105.h"
#include "spo2_algorithm.h"
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <freertos/semphr.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// ==================== PIN DEFINITIONS ====================
#define I2C_SDA 21
#define I2C_SCL 22
#define ONE_WIRE_BUS 4
#define MPU6050_ADDR 0x68
#define BUTTON_PIN 13
#define BUZZER_PIN 18

// ==================== WIFI & MQTT CONFIG ====================
const char* ssid = "iPhone";          // <-- CHANGE
const char* password = "koky1234";  // <-- CHANGE

// HiveMQ Public Broker
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;

// MQTT Topics
const char* topic_vitals = "guc/ward1/B/vitals";

// ==================== RTOS HANDLES ======================
TaskHandle_t TaskFallDetection;
TaskHandle_t TaskVitals;
TaskHandle_t TaskMQTT;
SemaphoreHandle_t i2cMutex;
SemaphoreHandle_t dataMutex;

// ==================== FALL DETECTION CONSTANTS ====================
const float FALL_THRESHOLD = 28.0f;
bool fallFlag = false;

bool fallMessageActive = false;
unsigned long fallTriggerTime = 0;
const unsigned long FALL_HOLD_TIME = 3000;
unsigned long lastButtonPress = 0;
const unsigned long DEBOUNCE_DELAY = 200;

// ==================== DS18B20 SETUP ====================
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature tempSensors(&oneWire);
DeviceAddress tempSensor;
float temperatureC = 0.0f;
bool tempValid = false;
unsigned long lastTempRequest = 0;
bool tempRequested = false;
```

```
// ==================== MPU6050 SETUP ====================
volatile float accelX = 0.0f, accelY = 0.0f, accelZ = 0.0f;
volatile float magnitude = 0.0f;
bool mpuValid = false;

// ==================== MAX30102 & DSP SETUP ====================
MAX30105 particleSensor;
uint32_t irBuffer[100];
uint32_t redBuffer[100];
int32_t bufferLength;
int32_t spo2 = 0;
int8_t validSPO2 = 0;
int32_t heartRate = 0;
int8_t validHeartRate = 0;
long irValue = 0;
bool maxValid = false;

// --- Low Pass Filter Variables ---
float alphaHR = 0.2f;
float alphaSpO2 = 0.2f;

float filteredHR = 0.0f;
float filteredSpO2 = 0.0f;

bool firstHR = true;
bool firstSpO2 = true;
int smoothedHeartRate = 0;

// ==================== MQTT CLIENT ====================
WiFiClient espClient;
PubSubClient mqttClient(espClient);

// ==================== TIMING ====================
unsigned long lastDisplayTime = 0;
const unsigned long displayInterval = 1000;
unsigned long lastMQTTSend = 0;
const unsigned long mqttInterval = 1000;

// ==================== FUNCTION PROTOTYPES ====================
void fallDetectionTask(void * parameter);
void vitalsTask(void * parameter);
void mqttTask(void * parameter);
void connectWiFi();
void reconnectMQTT();
void initMPU6050();
void initDS18B20();
void initMAX30102();
void readMPU6050();
void detectFall();
void readDS18B20();
void readMAX30102();
void publishSensorData();
void displayAllReadings();

// ==================== SETUP ====================
```

```
void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLDOWN);

  Serial.println("\n╔═══════════════════════════════════════════════════╗");
  Serial.println("║     ESP32 HEALTH & FALL MONITOR SYSTEM            ║");
  Serial.println("╚═══════════════════════════════════════════════════╝\n");

  Wire.begin(I2C_SDA, I2C_SCL);
  Wire.setClock(400000);

  i2cMutex = xSemaphoreCreateMutex();
  dataMutex = xSemaphoreCreateMutex();

  initMPU6050();
  initDS18B20();
  initMAX30102();

  connectWiFi();
  mqttClient.setServer(mqtt_server, mqtt_port);

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  delay(1000);

  xTaskCreatePinnedToCore(fallDetectionTask, "FallTask", 10000, NULL, 2,
&TaskFallDetection, 0);
  xTaskCreatePinnedToCore(vitalsTask, "VitalsTask", 10000, NULL, 1,
&TaskVitals, 1);
  xTaskCreatePinnedToCore(mqttTask, "MQTTTask", 8192, NULL, 1, &TaskMQTT,
1);
}

// ==================== MAIN LOOP ====================
void loop() {
  if (millis() - lastDisplayTime >= displayInterval) {
    displayAllReadings();
    lastDisplayTime = millis();
  }
  vTaskDelay(pdMS_TO_TICKS(100));
}

// ==================== WIFI ====================
void connectWiFi() {
  Serial.print("Connecting to WiFi");
  WiFi.begin(ssid, password);
  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 20) {
    delay(500);
    Serial.print(".");
    attempts++;
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\n✓ WiFi connected. IP: " +
WiFi.localIP().toString());
```

```
  } else {
    Serial.println("\n✗ WiFi connection failed. Will retry in task.");
  }
}

// ==================== MQTT RECONNECT ====================
void reconnectMQTT() {
  while (!mqttClient.connected()) {
    Serial.print("Attempting MQTT connection to HiveMQ...");
    String clientId = "ESP32_HealthMonitor_" + String(random(0xffff),
HEX);
    if (mqttClient.connect(clientId.c_str())) {
      Serial.println(" connected");
    } else {
      Serial.print("failed, rc=");
      Serial.print(mqttClient.state());
      Serial.println(" try again in 2 seconds");
      vTaskDelay(pdMS_TO_TICKS(2000));
    }
  }
}

// ==================== MQTT TASK ====================
void mqttTask(void * parameter) {
  for (;;) {
    if (WiFi.status() != WL_CONNECTED) {
      Serial.println("WiFi disconnected. Reconnecting...");
      WiFi.disconnect();
      WiFi.reconnect();
      vTaskDelay(pdMS_TO_TICKS(5000));
      continue;
    }

    if (!mqttClient.connected()) {
      reconnectMQTT();
    }
    mqttClient.loop();

    if (millis() - lastMQTTSend >= mqttInterval) {
      publishSensorData();
      lastMQTTSend = millis();
    }
    vTaskDelay(pdMS_TO_TICKS(50));
  }
}

// ==================== SENSOR TASKS ====================
void fallDetectionTask(void * parameter) {
  for (;;) {
    readMPU6050();
    detectFall();

    if (digitalRead(BUTTON_PIN) == HIGH) {
      unsigned long now = millis();
      if (now - lastButtonPress > DEBOUNCE_DELAY) {
        lastButtonPress = now;
```

```
        if (magnitude < (FALL_THRESHOLD * 0.5)) {
           if (xSemaphoreTake(dataMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
              fallFlag = false;
              fallMessageActive = false;

              // --- BUZZER SILENCE ---
              digitalWrite(BUZZER_PIN, LOW);

              xSemaphoreGive(dataMutex);
              Serial.println("✅ Fall flag cleared by physical
button.");
           }
        }
      }
    }
    vTaskDelay(pdMS_TO_TICKS(10));
  }
}

void vitalsTask(void * parameter) {
  for (;;) {
    readMAX30102();
    readDS18B20();
    vTaskDelay(pdMS_TO_TICKS(10));
  }
}

// ==================== SENSOR INIT ====================
void initMPU6050() {
  Serial.print("MPU6050: ");
  Wire.beginTransmission(MPU6050_ADDR);
  Wire.write(0x6B);
  Wire.write(0x00);
  if (Wire.endTransmission() == 0) {
    mpuValid = true;
    Serial.println("✓ Connected");
  } else {
    mpuValid = false;
    Serial.println("✗ Not found");
  }
}

void initDS18B20() {
  Serial.print("DS18B20: ");
  tempSensors.begin();
  if (tempSensors.getDeviceCount() > 0) {
    tempSensors.getAddress(tempSensor, 0);
    tempSensors.setResolution(tempSensor, 12);
    tempSensors.setWaitForConversion(false);
    tempValid = true;
    Serial.println("✓ Connected");
  } else {
    tempValid = false;
    Serial.println("✗ Not found");
```

```
  }
}

void initMAX30102() {
  Serial.print("MAX30102: ");
  if (particleSensor.begin(Wire, I2C_SPEED_FAST)) {
    maxValid = true;
    particleSensor.setup(0x1F, 4, 2, 100, 411, 4096);
    firstHR = true;
    firstSpO2 = true;
    Serial.println("✓ Connected");
  } else {
    maxValid = false;
    Serial.println("✗ Not found");
  }
}

// ==================== READ SENSORS ====================
void readMPU6050() {
  if (!mpuValid) return;
  if (xSemaphoreTake(i2cMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(0x3B);
    if (Wire.endTransmission(false) != 0) {
      xSemaphoreGive(i2cMutex);
      return;
    }
    Wire.requestFrom(MPU6050_ADDR, 6, true);
    if (Wire.available() == 6) {
      int16_t ax = (Wire.read() << 8) | Wire.read();
      int16_t ay = (Wire.read() << 8) | Wire.read();
      int16_t az = (Wire.read() << 8) | Wire.read();
      xSemaphoreGive(i2cMutex);
      accelX = ax / 16384.0f;
      accelY = ay / 16384.0f;
      accelZ = az / 16384.0f;
      magnitude = sqrt(accelX*accelX + accelY*accelY + accelZ*accelZ) *
9.8f;
    } else {
      xSemaphoreGive(i2cMutex);
    }
  }
}

void detectFall() {
  if (!mpuValid) return;
  if (magnitude > FALL_THRESHOLD && !fallMessageActive) {
    if (xSemaphoreTake(dataMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
      fallMessageActive = true;
      fallTriggerTime = millis();

       // --- BUZZER ACTIVATION ---
      digitalWrite(BUZZER_PIN, HIGH);

      xSemaphoreGive(dataMutex);
      Serial.println("\n!!! FALL DETECTED !!!");
```

```
      Serial.print("Magnitude: "); Serial.println(magnitude);
    }
  }
  fallFlag = fallMessageActive;
}

void readDS18B20() {
  if (!tempValid) return;

  // 1. If we haven't requested a temperature yet, request one and start
the clock.
  if (!tempRequested) {
    tempSensors.requestTemperatures();
    lastTempRequest = millis();
    tempRequested = true;
  }
  // 2. If we HAVE requested one, check if 750ms has passed for the 12-
bit conversion.
  else {
    if (millis() - lastTempRequest >= 750) {
      float temp = tempSensors.getTempCByIndex(0);

      // Basic sanity check to ensure it's not a dummy value or
disconnect error
      if (temp != DEVICE_DISCONNECTED_C && temp > -127 && temp != 0.00f)
{
        if (xSemaphoreTake(dataMutex, pdMS_TO_TICKS(10)) == pdTRUE) {
          temperatureC = temp;
          xSemaphoreGive(dataMutex);
          // Serial.println("temp has been read"); // Uncomment to debug
        }
      }

      // Reset the flag so we can request a new reading on the next loop
      tempRequested = false;
    }
  }
}

void readMAX30102() {
  if (!maxValid) return;
  bufferLength = 100;
  for (byte i = 0; i < bufferLength; i++) {
    unsigned long startWait = millis();
    const unsigned long SENSOR_TIMEOUT = 2000;
    while (true) {
      bool hasData = false;
      if (xSemaphoreTake(i2cMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
        particleSensor.check();
        hasData = particleSensor.available();
        xSemaphoreGive(i2cMutex);
      }
      if (hasData) break;
      if (millis() - startWait > SENSOR_TIMEOUT) {
        maxValid = false;
        Serial.println("MAX30102 timeout — disabling sensor.");
```

```
        return;
      }
      vTaskDelay(pdMS_TO_TICKS(5));
    }
    redBuffer[i] = particleSensor.getRed();
    irBuffer[i] = particleSensor.getIR();
    particleSensor.nextSample();
  }

  maxim_heart_rate_and_oxygen_saturation(
    irBuffer, bufferLength, redBuffer,
    &spo2, &validSPO2, &heartRate, &validHeartRate
  );

  irValue = irBuffer[99];

  // ================ LOW PASS FILTER ================

  // Heart Rate Filtering
  if (validHeartRate && heartRate > 40 && heartRate < 180) {
    if (firstHR) {
      filteredHR = heartRate;  // initialize
      firstHR = false;
    } else {
      filteredHR = alphaHR * heartRate + (1 - alphaHR) * filteredHR;
    }
    smoothedHeartRate = (int)filteredHR;
  }

  // SpO2 Filtering
  if (validSPO2 && spo2 > 70 && spo2 <= 100) {
    if (firstSpO2) {
      filteredSpO2 = spo2;
      firstSpO2 = false;
    } else {
      filteredSpO2 = alphaSpO2 * spo2 + (1 - alphaSpO2) * filteredSpO2;
    }
    spo2 = (int)filteredSpO2;
  }
}

// ================== PUBLISH JSON OVER MQTT ==================
void publishSensorData() {
  float temp, mag;
  int hr, spo2_val;
  bool fallActive;
  bool tempOk, hrOk;

  if (xSemaphoreTake(dataMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
    temp = temperatureC;
    mag = magnitude;
    hr = smoothedHeartRate;
    spo2_val = spo2;
    fallActive = fallMessageActive;
    tempOk = tempValid;
    hrOk = maxValid && (irValue > 50000) && validHeartRate && !firstHR;
```

```
    xSemaphoreGive(dataMutex);
  } else {
    return;
  }

  StaticJsonDocument<256> doc;
  doc["timestamp"] = millis();
  doc["bpm"] = hrOk ? hr : -1;
  doc["spo2"] = (maxValid && validSPO2 && !firstSpO2) ? spo2_val : -1;
  doc["temp_c"] = tempOk ? temp : -999.0f;
  doc["accel"] = JsonObject();
  doc["accel"]["x"] = accelX;
  doc["accel"]["y"] = accelY;
  doc["accel"]["z"] = accelZ;
  doc["magnitude"] = mag;
  doc["fall_active"] = fallActive;

  String jsonString;
  serializeJson(doc, jsonString);

  if (mqttClient.connected()) {
    mqttClient.publish(topic_vitals, jsonString.c_str());
    Serial.println("[MQTT] Published: " + jsonString);
  }
}

// ==================== DISPLAY ====================
void displayAllReadings() {
  float temp, mag;
  int hr, spo2_val;
  bool fallActive, tempOk, hrOk;

  if (xSemaphoreTake(dataMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
    temp = temperatureC;
    mag = magnitude;
    hr = smoothedHeartRate;
    spo2_val = spo2;
    fallActive = fallMessageActive;
    tempOk = tempValid;
    hrOk = maxValid && (irValue > 50000);
    xSemaphoreGive(dataMutex);
  } else {
    return;
  }

  Serial.println("\n┌─────────────────────────────────────────┐");
  Serial.println("│           MONITOR STATUS REPORT         │");
  Serial.println("├─────────────────────────────────────────┤");
  Serial.print("│ ⚠️  Fall Status:    ");
  if (fallActive) {
    Serial.println("⚠️  FALL DETECTED! ⚠️ ");
  } else if (mag > 20.0f) {
    Serial.println("HIGH IMPACT DETECTED!");
  } else {
    Serial.println("System Normal");
```

```
  }
  Serial.print("│ 🌡️  Temperature:    ");
  if (tempOk) {
    Serial.print(temp, 1); Serial.println(" °C");
  } else {
    Serial.println("ERROR");
  }
  Serial.print("│ 📐 Magnitude:      ");
  Serial.print(mag, 2); Serial.println(" m/s^2");
  Serial.println("│ ❤️  Health Monitor:");
  if (hrOk) {
    Serial.print("│    HR: "); Serial.print(hr); Serial.print(" BPM │
");
    Serial.print("SpO2: "); Serial.print(spo2_val); Serial.println("%");
  } else {
    Serial.println("│    Place finger on sensor...");
  }
  Serial.println("└──────────────────────────────────────────────────┘");
}
```